Apresentação Final

Roteamento Dinâmico de Consultas SQL

Baseado em Metadados de Tabelas Apache Iceberg

Aristides Henrique Gonçalves da Cruz

Sistemas de Informação
ICEI, PUC Minas, Unidade São Gabriel
arihenriquedev@hotmail.com

Outubro de 2026

Trabalho: Trabalho de Conclusão de Curso
Orientador: Prof. Gustavo Luís Soares

PUC Minas

Contexto

- Volume crescente em *data lakes* e *lakehouses* exige melhor aproveitamento no consumo dos dados por consulta
- *Apache Iceberg* como formato de tabela com metadados hierárquicos
- *Trino* como *engine* de consultas distribuído
- Em geral, o *cluster* de execução é escolhido de forma estática

Problema

Consultas leves ocupam *clusters* robustos e consultas pesadas falham ou degradam em *clusters* pequenos, com aproveitamento ineficiente dos recursos de processamento.

PUC Minas

Motivação

- Provisionamento excessivo eleva custos operacionais e financeiros, além de desperdiçar recursos computacionais
- Subdimensionamento aumenta falhas e tempo de resposta
- Metadados do *Iceberg* permitem estimar o custo **antes** da execução
- Lacuna: roteamento entre *clusters Trino* de capacidades distintas

EX CORDE ECCLESIAE
PUC Minas

Objetivos

Objetivo Geral

Desenvolver o *DyraSQL*, mecanismo que usa *EXPLAIN (TYPE IO)* para definir automaticamente o *cluster* mais adequado para cada consulta SQL.

1. Mapear metadados *Iceberg* úteis para estimar custo
2. Definir regras de decisão
3. Construir protótipo local
4. Realizar experimentos controlados

PUC Minas

Referencial Teórico

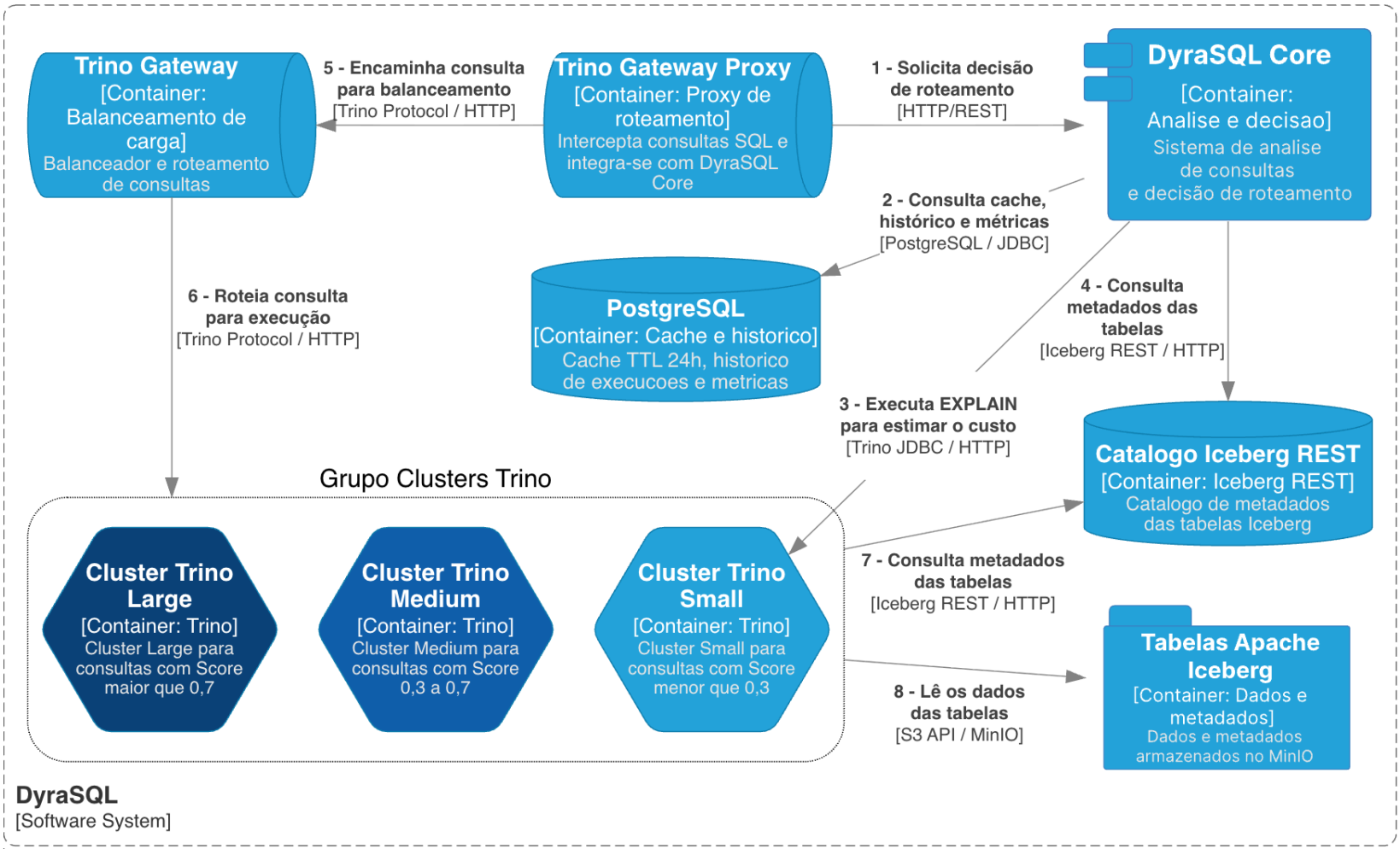
- *Apache Iceberg*: *snapshots*, manifestos e estatísticas por arquivo (*data skipping*)
- *Trino*: *EXPLAIN (TYPE IO)* retorna *outputSizeInBytes* e linhas estimadas
- *Trino Gateway*: balanceamento e roteamento entre *clusters*
- *Lakehouse*: unificação de *data lake* e *warehouse*

Trabalhos relacionados

Strausz et al. (2025) selecionam o *engine* SQL. O *DyraSQL* mantém o mesmo *engine* (*Trino*) e seleciona o *cluster* com o dimensionamento mais adequado (*small*, *medium* ou *large*).

PUC Minas

Metodologia - Arquitetura



Fonte: Elaborado pelo autor.

Metodologia - Algoritmo de Decisão

Score

$$S = w_1 f_v + w_2 f_c + w_3 f_h \tag{1}$$

- f_v: volume efetivo da consulta
- f_c: complexidade estrutural
- f_h: histórico de execuções
- w_1=0,5, w_2=0,3, w_3=0,2 nos testes, ajustáveis ao ambiente

Roteamento

$S < 0{,}3$: *small* $0{,}3 \leq S \leq 0{,}7$: *medium* $S > 0{,}7$: *large*

Decisões em *cache* por *fingerprint*, com *TTL* de 24 horas.



Metodologia - Exemplo de Cálculo

Fator volume

$$f_v = \left(\frac{\log(T_e)}{\log(T_{max})} \times 0{,}6 + \frac{\log(R_e)}{\log(R_{max})} \times 0{,}4 \right) \times (1 - F_o)$$

- $T_e = 1{,}25$ GB e $R_e = 2{,}5 \times 10^6$ vêm do *EXPLAIN (TYPE IO)* da consulta
- T_{max} e R_{max} são o volume máximo do *warehouse* de cada ambiente (no exemplo, 1 TB e 10^9 linhas)

$$f_v = \left(\frac{\log(1{,}25)}{\log(1000)} \times 0{,}6 + \frac{\log(2500000)}{\log(10^9)} \times 0{,}4 \right) \times 0{,}9 = 0{,}27$$

PUC Minas

Metodologia - Exemplo de Cálculo

Fator complexidade

$$f_c = \frac{1 \times 0{,}2 + 3 \times 0{,}15 + 2 \times 0{,}25 + 1 \times 0{,}02 + 2 \times 0{,}1}{2{,}0} = 0{,}69$$

- 1 *JOIN* (0,2), 3 agregações (0,15), 2 subconsultas (0,25)
- 1 filtro particionado (0,02) e 2 não particionados (0,1), soma ponderada 1,37
- L_c=2 porque a soma 1,37 não pode saturar em 1, nem perder peso com teto grande

Fator histórico

- sem histórico, $f_h = 0{,}5$ (caso do exemplo, neutro)
- sucesso, $f_h = S_p$ (repete a faixa que funcionou)
- falha em *small*/*medium*, $f_h = 1 - S_p$ (busca mais capacidade)
- falha no *large*, $f_h = 0{,}5$ (não reduz, não é subdimensionamento)

Resultados - Ambiente em Execução

PRINT PENDENTE

AMBIENTE.PNG

DOCKER COMPOSE PS

Fonte: Elaborado pelo autor. Saída de docker compose ps.

Resultados - Dados no *Iceberg*



Fonte: Elaborado pelo autor. Consulta ao dataset sintético.

Resultados - *EXPLAIN (TYPE IO)*



Fonte: Elaborado pelo autor. Metadados extraídos antes da execução.

PUC Minas

Resultados - Decisões de Roteamento

PRINT PENDENTE

ROTEAMENTO.PNG

SCORE E CLUSTER

Fonte: Elaborado pelo autor. Score e *cluster* escolhido por consulta.

PUC Minas

Resultados - *Cache* de Decisões

PRINT PENDENTE

CACHE.PNG

CACHE MISS E HIT

Fonte: Elaborado pelo autor. *Cache miss* e *cache hit*.

PUC Minas

Resultados - Síntese

Consulta de referência

$$S = 0{,}5 \times 0{,}27 + 0{,}3 \times 0{,}69 + 0{,}2 \times 0{,}5 = 0{,}44$$

Volume f_v=0,27, complexidade f_c=0,69 e histórico f_h=0,5. Com $0{,}3 \leq S \leq 0{,}7$, a consulta segue para o *cluster medium*.

- Consultas simples e com filtros estreitos tendem ao *cluster small*
- *EXPLAIN (TYPE IO)* fornece volume compatível com os filtros de partição
- Reexecuções recuperam a decisão do *cache*

Limitação: *dataset* sintético, sem elasticidade de nuvem.

Cronograma

Situação atual

Artigo, protótipo e experimentos do *DyraSQL* estão concluídos.

- Preparar a apresentação da banca final
- Ensaio da defesa com o orientador
- Entrega da versão final no Canvas e no Pergamum

PUC Minas

Obrigado!

Aristides Henrique Gonçalves da Cruz
Orientador: Prof. Gustavo Luís Soares
arihenriquedev@hotmail.com

